Test harnesses

CSC103 Programming Fundamentals / Lecture 30

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Create a harness for absolute value over -5, 0 and 5. State what your method does for Integer.MIN_VALUE.

Store manually calculated expected results in long values.

Learning objectives

- Build a repeatable test harness
- Choose expected results independently
- Report passes and failures meaningfully

CLO-4
Tests and oracles
A simple harness
Equivalence and boundaries
Repeatability and regression

Tests and oracles

- A test supplies input and checks observable behaviour.
- An oracle specifies the expected outcome.
- Calculate expected results independently of the implementation.

Example

```
Function: max(a,b)
Input: (-4,-2)
Expected: -2
Observed: result returned by the method
```

A simple harness

- A harness calls the target repeatedly and reports results.
- Each failure should identify its case and expected value.
- A summary helps detect missed or skipped cases.

Example

```
Case: negative pair
Expected: -2
Actual: 0
Status: FAIL
```

Equivalence and boundaries

- Group inputs that should behave similarly.
- Test transitions between groups and exceptional inputs.
- Include empty or zero-sized input where the contract permits it.

Example

```
For isPass(mark):
49, 50, 51
For a valid-mark contract:
-1, 0, 100, 101
```

Repeatability and regression

- Tests should produce the same result under the same conditions.
- Control random seeds, data files and other changing dependencies.
- Keep a small test that exposes each fixed bug.

Example

```
Regression case:
Input: two negative numbers
Expected: larger negative number
Purpose: detect zero initialisation bug
```

The expected result must be independent

- A harness compares actual behaviour to an oracle derived from the specification.
- Using the same potentially faulty implementation on both sides proves little.
- Literal expected results work well for a small set of manually solved cases.

Example

```
Input -5 has expected absolute value 5.
The expected value comes from the mathematical definition.
```

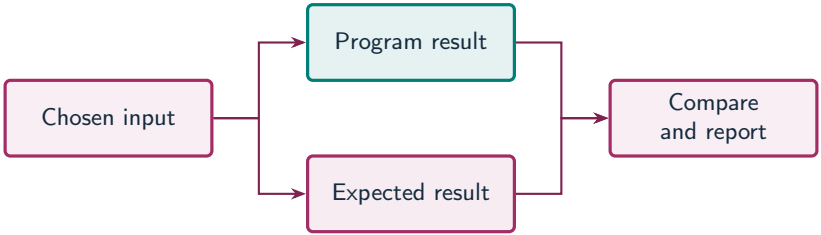
Numeric limits belong in the contract

- Integer.MIN_VALUE has no positive counterpart representable as int.
- Returning long lets an absolute-value function represent that magnitude.
- The conversion must happen before negation to avoid int overflow.

Example

```
long widened = value;
return widened < 0 ? -widened : widened;
```

A test harness compares two independent paths



What to notice

The expected result must come from the specification, not from calling the same implementation again.

Key distinctions

Input	Expected long result	Reason for case
-5	5	Negative ordinary input
0	0	Zero boundary
5	5	Positive ordinary input
Integer.MIN_VALUE	2147483648	Minimum int magnitude

These distinctions determine which operation or control structure fits the problem.

First example: methods

```
static int max(int a, int b) { return a >= b ? a : b; }
```

First example: execution

```
int[][] cases = {{2, 5, 5}, {-4, -2, -2}, {3, 3, 3}};
int passed = 0;
for (int[] c : cases) {
  int actual = max(c[0], c[1]);
  if (actual == c[2]) passed++;
  else System.out.println("FAIL expected " + c[2] + " got " + actual);
}
System.out.println("Passed " + passed + "/" + cases.length);
```

First example: result

Passed 3/3
Each row contains first input, second input and expected result.
The equal-input case checks ties.

Equivalence and boundaries
Repeatability and regression

Guided practice

Create a harness for absolute value over -5, 0 and 5. State what your method does for Integer.MIN_VALUE.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Store manually calculated expected results in long values.
- Call the absolute-value method for each input.
- Print expected and actual values with a clear pass or fail label.

The next program uses fixed inputs so its result can be reproduced.

Complete solution (1/2)

```
public class Solution30 {
  public static void main(String[] args) throws Exception {
    int[] inputs = {-5, 0, 5, Integer.MIN_VALUE};
    long[] expected = {5L, 0L, 5L, 2147483648L};
    for (int i = 0; i < inputs.length; i++) {
      long actual = absolute(inputs[i]);
      String status = actual == expected[i] ? "PASS" : "FAIL";
      System.out.println(status + " " + inputs[i] + " => " + actual);
```

Complete solution (2/2)

```
    }
  }
  static long absolute(int value) {
    long widened = value;
    return widened < 0 ? -widened : widened;
  }
}
```

Execution trace

Input	Widened value	Negation needed?	Actual result
-5	-5L	Yes	5L
0	0L	No	0L
5	5L	No	5L
Minimum int	-2147483648L	Yes	2147483648L

Follow the changing state in execution order.

Solution result

PASS -5 => 5
PASS 0 => 0
PASS 5 => 5
PASS -2147483648 => 2147483648

Why this result follows
Print expected and actual values
with a clear pass or fail label.

A common incorrect approach

```
int expected = buggyMax(a,b);
int actual = buggyMax(a,b);
```

Example

The oracle repeats the same implementation. Write the expected value from the contract and a manually solved case.

Boundary and failure checks

Check the stated input assumptions.
Build a harness with at least six cases for your average or maximum method and retain one deliberate bug demonstration.

Expected ordinary results: 5,0,5. Since the positive counterpart of Integer.MIN_VALUE does not fit int, explicitly reject it, use long, or document the behaviour. Do not invent an in-range result.

Check your understanding

What makes a failed test useful? Why include a tie case for maximum?

Write a prediction and explain the rule that supports it.

Answer and explanation

It names the case and reports expected versus actual. Ties exercise equality rather than only strict ordering.

The oracle repeats the same implementation. Write the expected value from the contract and a manually solved case.

Compare cases and predict outcomes

Test	Expected max	Fault it can expose
max(3,7)	7	Wrong comparison
max(-3,-7)	-3	Incorrect zero initialisation
max(5,5)	5	Mishandled equality

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Build a small explicit check for max(-3,-7). Explain why Java assert statements can be an unreliable default harness without the required runtime option.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
int actual = Math.max(-3, -7);
int expected = -3;
if (actual != expected) {
  throw new AssertionError("expected -3, got " + actual);
}
System.out.println("Passed");
```

- The expected value is reasoned from the ordering of negative numbers.
- The explicit check executes normally.
- Java language assertions are disabled by default unless enabled, for example with -ea.

Lecture summary

- a repeatable test harness
- expected results independently
- Report passes and failures meaningfully

- Store manually calculated expected results in long values.
- Call the absolute-value method for each input.
- Print expected and actual values with a clear pass or fail label.

After-class practice

Build a harness with at least six cases for your average or maximum method and retain one deliberate bug demonstration.

Syllabus reading
Reference material
Next lecture: Unit testing with JUnit